Configuration (.cfg) File Format

Each line in the .cfg file describes a command with parameters. The various commands and their arguments are described in the table below (arguments are in sequence). For mmW demo, users can create their own config files from the Visualizer GUI by using the "Save Config to PC" button or starting from the few sample profiles provided in the mmwave_sdk_<ver>\packages\ti\demo\<platform>\mmw\profiles directory.

Converting configuration from older SDK release to current SDK release

As new versions of SDK releases are available, there are usually changes to the configuration commands that are supported in the new release. Now, users may have some hand crafted config file which worked perfectly well on older SDK release version but will not work as is with the new SDK release. If user desires to run the same configuration against the new SDK release, then there is a script mmwDemo_<platform>_update_config.pl provided in the mmwave_sdk_<ver>\packages\ti\demo\<platform>\mmw\profiles directory that they can use to convert the configuration file from older release to a compatible version for the new release. Refer to the perl file for details on how to run the script. Note that users will need to install perl on their machine (there is no strict version requirement at this time). For any new commands inserted by the script, there will be a comment preceeding that line which is similar to something like this: "Inserting new mandatory command. Check users guide for details."

Most of the parameters described below are the same as the mmwavelink API specifications (see doxygen mmwave_sdk_<ver>\packages\ti\control\mmwavelink\docs\doxygen\html\index.html.) Additionally, users can refer to the chirp diagram below to understand the chirp and profile related parameters or the appnote http://www.ti.com/litv/pdf/swra553

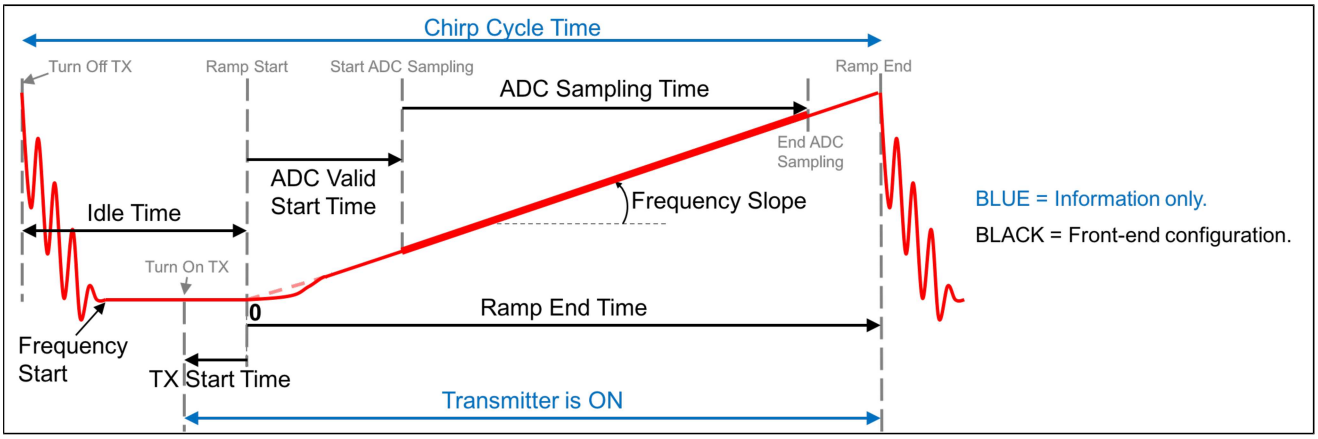

Figure 2: Chirp Diagram

Configuration command	Command details	Command Parameters	Usage in mmW demo xwr16xx/xwr18xx /xwr64xx/xwr68xx
dfeDataOutputMode	The values in this command should not change between sensorStop and sensorStart. Reboot the board to try config with different set of values in this command This is a mandatory command.	<modeType> 1 - frame based chirps 2 - continuous chirping 3 - advanced frame config	only option 1 and 3 are supported
channelCfg	Channel config message to RadarSS. See mmwavelink doxgen for details. The values in this command should not change between sensorStop and sensorStart. Reboot the board to try config with different set of values in this command This is a mandatory command.	<rxChannelEn> Receive antenna mask e.g for 4 antennas, it is 0x1111b = 15	4 antennas supported

		<txChannelEn> Transmit antenna mask	Refer to the antenna layout on the EVM /board to determine the right Tx antenna mask needed to enable the desired virtual antenna configuration. For example, in IWR6843 ISK, the 2 azimuth antennas can be enabled using bitmask 0x5 (i.e. tx1 and tx3). The azimuth and elevation antennas can both be enabled using bitmask 0x7 (i.e. tx1, tx2 and tx3). For example, in xWR1642BOOST, the azimuth antennas can be enabled using bitmask 0x3 (i.e. tx1 and tx2).
		<cascading> SoC cascading, not applicable, set to 0	n/a
adcCfg	ADC config message to RadarSS. See mmwavelink doxgen for details. The values in this command should not change between sensorStop and sensorStart. Reboot the board to try config with different set of values in this command This is a mandatory command.		
		<numADCBits> Number of ADC bits (0 for 12-bits, 1 for 14-bits and 2 for 16-bits)	only 16-bit is supported
		<adcOutputFmt> Output format : 0 - real 1 - complex 1x (image band filtered output) 2 - complex 2x (image band visible))	only complex modes are supported
adcbufCfg	adcBuf hardware config. The values in this command can be changed between sensorStop and sensorStart. This is a mandatory command.		
		<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 For advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
		<adcOutputFmt> ADCBUF out format 0-Complex, 1-Real	only complex modes are supported
		<SampleSwap> ADCBUF IQ swap selection: 0-I in LSB, Q in MSB, 1-Q in LSB, I in MSB	only option 1 is supported
		<ChanInterleave> ADCBUF channel interleave configuration: 0 - interleaved(only supported for XWR14xx), 1 - non-interleaved	only option 1 is supported

		<ChirpThreshold> Chirp Threshold configuration used for ADCBUF buffer to trigger ping /pong buffer switch. Valid values: 0-8 for demos that use DSP for 1D FFT and LVDS streaming is disabled only 1 for demos that use HWA for 1D FFT	xwr16xx demo: Values 0-8 are supported since it uses DSP for 1D FFT However, only value of 1 is supported when LVDS streaming is enabled. xwr64xx/xwr68xx /xwr18xx: only value of 1 is supported since these demos use HWA for 1D FFT
profileCfg	Profile config message to RadarSS and datapath. See mmwavelink doxgen for details. The values in this command can be changed between sensorStop and sensorStart. This is a mandatory command. **txCalibEnCfg Field** This CLI command doesn't expose the txCalibEnCfg field in the mmwavelink structure. User should follow the mmwavelink documentation and update the CLI profileCfg handler function accordingly. The current handler sets the value to 0 for this field (backward compatible mode) Combination of numAdcSamples in profileCfg (and numRangeBins), numDopplerChirps = total number of chirps/(num TX in MIMO mode) in frameCfg or subFrameCfg, number of TX and RX antennas in channelCfg and chirpCfg determine the size of Radarcube and other internal buffers/heap in the demo. It is possible that some combinations of these values result in out of memory conditions for these heaps and demo will reject such configuration. Refer to demo and DPC doxygen to understand the data buffer layout and use the system printfs on sensorStart in CCS console window to understand the exact heap usage for a given configuration.		
		<profileId> profile Identifier	Legacy frame (dfeOutputMode=1): could be any allowed value but only one valid profile per config is supported Advanced frame (dfeOutputMode=3): could be any allowed value but only one profile per subframe is supported. However, different subframes can have different profiles
		<startFreq> "Frequency Start" in GHz (float values allowed) Examples: 77 61.38	any value as per mmwavelink doxgen /device datasheet but represented in GHz. Refer to the chirp diagram shown above to understand the relation between various profile parameters and inter-dependent constraints.
		<idleTime> "Idle Time" in u-sec (float values allowed) Examples: 7 7.15	any value as per mmwavelink doxgen /device datasheet but represented in usec. Refer to the chirp diagram shown above to understand the relation between various profile parameters and inter-dependent constraints.
		<adcStartTime> "ADC Valid Start Time" in u-sec (float values allowed) Examples: 7 7.34	any value as per mmwavelink doxgen /device datasheet but represented in usec. Refer to the chirp diagram shown above to understand the relation between various profile parameters and inter-dependent constraints.

<rampEndTime> "Ramp End Time" in u-sec (float values allowed) Examples: 58 216.15	any value as per mmwavelink doxgen /device datasheet but represented in usec Refer to the chirp diagram shown above to understand the relation between various profile parameters and inter-dependent constraints.
<txOutPower> Tx output power back-off code for tx antennas	only value of '0' has been tested within context of mmW demo
<txPhaseShifter> tx phase shifter for tx antennas	only value of '0' has been tested within context of mmW demo
<freqSlopeConst> "Frequency slope" for the chirp in MHz/usec (float values allowed) Examples: 68 16.83	any value greater than as per mmwavelink doxgen/device datasheet but represented in MHz /usec. Refer to the chirp diagram shown above to understand the relation between various profile parameters and inter-dependent constraints.
<txStartTime> "TX Start Time" in u-sec (float values allowed) Examples: 1 2.92	any value as per mmwavelink doxgen /device datasheet but represented in usec. Refer to the chirp diagram shown above to understand the relation between various profile parameters and inter-dependent constraints.
<numAdcSamples> number of ADC samples collected during "ADC Sampling Time" as shown in the chirp diagram above Examples: 256 224	any value as per mmwavelink doxgen /device datasheet. Refer to the chirp diagram shown above to understand the relation between various profile parameters and inter-dependent constraints.
<digOutSampleRate> ADC sampling frequency in ksps. (<numAdcSamples> / <digOutSampleRate> = "ADC Sampling Time") Examples: 5500	any value as per mmwavelink doxgen /device datasheet. Refer to the chirp diagram shown above to understand the relation between various profile parameters and inter-dependent constraints.

		<hpfCornerFreq1> HPF1 (High Pass Filter 1) corner frequency 0: 175 KHz 1: 235 KHz 2: 350 KHz 3: 700 KHz	any value as per mmwavelink doxgen /device datasheet
		<hpfCornerFreq2> HPF2 (High Pass Filter 2) corner frequency 0: 350 KHz 1: 700 KHz 2: 1.4 MHz 3: 2.8 MHz	any value as per mmwavelink doxgen /device datasheet
		<rxGain> OR'ed value of RX gain in dB and RF gain target (See mmwavelink doxgen for details)	any value as per mmwavelink doxgen /device datasheet
chirpCfg	Chirp config message to RadarSS and datapath. See mmwavelink doxgen for details. The values in this command can be changed between sensorStop and sensorStart. This is a mandatory command.		
		chirp start index	any value as per mmwavelink doxygen
		chirp end index	any value as per mmwavelink doxygen
		profile identifier	should match the profileCfg->profileId
		start frequency variation in Hz (float values allowed)	only value of '0' has been tested within context of mmW demo
		frequency slope variation in kHz/us (float values allowed)	only value of '0' has been tested within context of mmW demo
		idle time variation in u-sec (float values allowed)	only value of '0' has been tested within context of mmW demo
		ADC start time variation in u-sec (float values allowed)	only value of '0' has been tested within context of mmW demo
		tx antenna enable mask (Tx2,Tx1) e.g (10)b = Tx2 enabled, Tx1 disabled.	See note under "Channel Cfg" command above. Individual chirps should have either only one distinct Tx antenna enabled (MIMO) or same TX antennas should be enabled for all chirps
lowPower	Low Power mode config message to RadarSS. See mmwavelink doxgen for details. The values in this command should not change between sensorStop and sensorStart. Reboot the board to try config with different set of values in this command. This is a mandatory command.		
		<don't_care>	set to 0
		ADC Mode 0x00 : Regular ADC mode 0x01 : Low power ADC mode (Not supported for xwr6xxx devices)	use value of '0' or '1' (depending on profileCfg->digOutSampleRate)
frameCfg	frame config message to RadarSS and datapath. See mmwavelink doxgen for details. dfeOutputMode should be set to 1 to use this command The values in this command can be changed between sensorStop and sensorStart. This is a mandatory command when dfeOutputMode is set to 1.		
		chirp start index (0-511)	any value as per mmwavelink doxgen bu corresponding chirpCfg should be defined
		chirp end index (chirp start index-511)	any value as per mmwavelink doxgen bu corresponding chirpCfg should be defined

		number of loops (1 to 255)	any value as per mmwavelink doxgen /device datasheet but greater than or equal tc 4. For xwr16xx/xwr68xx demos where DSP version of Doppler DPL is used, the Doppler chirps (i.e. number of loops) should be a multiple of 4 due to windowing requirement Note: If value of 2 is desired for number of Doppler Chirps, one must update the demo /object detection DPC source code to use rectangular window for Doppler DPU instead o Hanning window.
		number of frames (valid range is 0 to 65535, 0 means infinite)	any value as per mmwavelink doxgen
		frame periodicity in ms (float values allowed)	any value as per mmwavelink doxgen and represented in msec. However frame should not have more than 50% duty cycle (i. e. active chirp time should be <= 50% of frame period). Also it should allow enough time for selected UART output to be shipped ou (selections based on guiMonitor command) else demo will assert if the next frame start trigger is received from the front end and current frame is still ongoing. User can use the output of stats TLV to tune this parameter.
		trigger select 1: Software trigger 2: Hardware trigger.	only option for Software trigger is supported
		Frame trigger delay in ms (float values allowed)	any value as per mmwavelink doxgen and represented in msec.
advFrameCfg	Advanced config message to RadarSS and datapath. See mmwavelink doxgen for details. The dfeOutputMode should be set to 3 to use this command. See profile_advanced_subframe.cfg profile in the mmW demo profiles directory for example usage. The values in this command can be changed between sensorStop and sensorStart. This is a mandatory command when dfeOutputMode is set to 3.		
		<numOfSubFrames> Number of sub frames enabled in this frame	any value as per mmwavelink doxgen
		<forceProfile> Force profile	only value of 0 is supported
		<numFrames> Number of frames to transmit (1 frame = all enabled sub frames)	any value as per mmwavelink doxgen
		<triggerSelect> trigger select 1: Software trigger 2: Hardware trigger.	only option for Software trigger is supported
		<frameTrigDelay> Frame trigger delay in ms (float values allowed)	any value as per mmwavelink doxgen and represented in msec.
subFrameCfg	Subframe config message to RadarSS and datapath. See mmwavelink doxgen for details.		

	The dfeOutputMode should be set to 3 to use this command. See profile_advanced_subframe.cfg profile in the mmW demo profiles directory for example usage The values in this command can be changed between sensorStop and sensorStart. This is a mandatory command when dfeOutputMode is set to 3.	<subFrameNum> subframe Number for which this command is being given	value of 0 to RL_MAX_SUBFRAME 1
		<forceProfileIdx> Force profile index	ignored as <forceProfile> in advFrameCfg should b set to 0
		<chirpStartIdx> Start Index of Chirp	any value as per mmwavelink doxgen bu corresponding chirpCfg should be defined
		<numOfChirps> Num of unique Chirps per burst including start index	any value as per mmwavelink doxgen bu corresponding number of chirpCfg should be defined
		<numLoops> No. of times to loop through the unique chirps	any value as per mmwavelink doxgen bu greater than or equal to 4 For xwr16xx/xwr68xx demos where DSP version of Doppler DPU is used, the Doppler chirps (i.e. number of loops) should be a multiple of 4 due to windowing requirement Note: If value of 2 is desired for number of Doppler Chirps, one must update the demo /object detection DPC source code to use rectangular window for Doppler DPU instead o Hanning window.
		<burstPeriodicity> Burst periodicty in msec (float values allowed) and meets the criteria burstPeriodicity >= ((numLoops)* (Sum total of time duration of all unique chirps in that burst)) + InterBurstBlankTime	any value as per mmwavelink doxgen and represented in msec but subframe should not have more than 50% duty cycle and allow enough time for selected UART output to be shipped ou (selections based on guiMonitor command)
		<chirpStartIdxOffset> Chirp Start address increament for next burst	set it to 0 since demo supports only one burs per subframe
		<numOfBurst> Num of bursts in the subframe	set it to 1 since demo supports only one burs per subframe
		<numOfBurstLoops> Number of times to loop over the set of above defined bursts, in the sub frame	set it to 1 since demo supports only one burs per subframe
		<subFramePeriodicity> subFrame periodicty in msec (float values allowed) and meets the criteria subFramePeriodicity >= Sum total time of all bursts + InterSubFrameBlankTime	set to same as <burstPeriodicity> sinc demo supports only on burst per subframe
guiMonitor	Plot config message to datapath. The values in this command can be changed between sensorStop and sensorStart. This is a mandatory command.		
		All parameters below are flags (1 to enable and 0 to disable)	

<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 whereas for advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
<detected objects> 1 - enable export of point cloud (x,y, z,doppler) and point cloud sideinfo (SNR, noiseval) 2 - enable export of point cloud (x,y, z,doppler) 0 - disable	all values supported
<log magnitude range> 1 - enable export of log magnitude range profile at zero Doppler 0 - disable	all values supported
<noise profile> 1 - enable export of log magnitude noise profile 0 - disable	all values supported
<rangeAzimuthHeatMap> or <rangeAzimuthElevationHeatMap> range-azimuth or range-azimuth-elevation heat map related information <rangeAzimuthHeatMap> This output is provided only in demos that use AoA (legacy) DPU for AoA processing 1 - enable export of zero Doppler radar cube matrix, all range bins, all azimuth virtual antennas to calculate and display azimuth heat map. (The GUI computes the FFT of this to show heat map) 0 - disable < rangeAzimuthElevationHeatMap > This output is provided in demos that use AoA 2D DPU for AoA processing (ex: mmW demo for IWR6843AOP) 1 - enable export of zero Doppler radar cube matrix, all range bins, all virtual antennas to calculate and display azimuth heat map. (The GUI remaps the antenna symbols and computes the FFT of this stream to show azimuth heat map only). 0 - disable	all values supported
<rangeDopplerHeatMap> range-doppler heat map 1 - enable export of the whole detection matrix. Note that the frame period should be adjusted according to UART transfer time. 0 - disable	all values supported

		<statsInfo> statistics (CPU load, margins, device temperature readings, etc) 1 - enable export of stats data. 0 - disable	all values supported
cfarCfg	CFAR config message to datapath. The values in this command can be changed between sensorStop and sensorStart and even when the sensor is running. This is a mandatory command.		
		<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 whereas for advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
		<procDirection> Processing direction: 0 – CFAR detection in range direction 1 – CFAR detection in Doppler direction	all values supported; 2 separate commands need to be sent; one fo Range and other for doppler.
		<mode> CFAR averaging mode: 0 - CFAR_CA (Cell Averaging) 1 - CFAR_CAGO (Cell Averaging Greatest Of) 2 - CFAR_CASO (Cell Averaging Smallest Of)	all values supported
		<noiseWin> noise averaging window length: Length of the one sided noise averaged cells in samples Make sure 2*(noiseWIn+guardLen) <numRangeBins for range direction and 2*(noiseWIn+guardLen) <numDopplerBins for doppler direction.	supported
		<guardLen> one sided guard length in samples Make sure 2*(noiseWIn+guardLen) <numRangeBins for range direction and 2*(noiseWIn+guardLen) <numDopplerBins for doppler direction.	supported
		<divShift> Cumulative noise sum divisor expressed as a shift. Sum of noise samples is divided by 2^<divShift>. Based on <mode> and <noiseWin> , this value should be set as shown in next columns. The value to be used here should match the "CFAR averaging mode" and the "noise averaging window length" that is selected above. The actual value that is used for division (2^x) is a power of 2, even though the "noise averaging window length" samples may not have that restriction.	CFAR_CA: <divShift> = ceil($\log_2$(2 x <noiseWin>)) CFAR_CAGO/_CASO: <divShift> = ceil($\log_2$ (<noiseWin>)) In profile_2d.cfg, value of 3 means that the noise sum is divided by 2^3=8 to get the average of noise samples with window length of 8 samples in CFAR -CASO mode.
		cyclic mode or Wrapped around mode. 0- Disabled 1- Enabled	supported

		Threshold scale in dB using float representation. This is used in conjuntion with the noise sum divisor (say x). the CUT comparison for log input is: CUT > (Threshold scale converted from dB to Q8) + (noise sum / 2^x) For example: 15 10.75	Detection threshold is specified in dB scale. Maximum value allowe is 100dB
		peak grouping 0 - disabled 1 - enabled	supported
multiObjBeamForming	Multi Object Beamforming config message to datapath. This feature allows radar to separate reflections from multiple objects originating from the same range/Doppler detection. The procedure searches for the second peak after locating the highest peak in Azimuth FFT. If the second peak is greater than the specified threshold, the second object with the same range /Doppler is appended to the list of detected objects. The threshold is proportional to the height of the highest peak. The values in this command can be changed between sensorStop and sensorStart and even when the sensor is running. This is a mandatory command.		
		<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 whereas for advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
		<Feature Enabled> 0 - disabled 1 - enabled	supported
		<threshold> 0 to 1 – threshold scale for the second peak detection in azimuth FFT output. Detection threshold is equal to <thresholdScale> multiplied by the first peak height. Note that FFT output is magnitude squared.	supported
calibDcRangeSig	DC range calibration config message to datapath. Antenna coupling signature dominates the range bins close to the radar. These are the bins in the range FFT output located around DC. When this feature is enabled, the signature is estimated during the first N chirps, and then it is subtracted during the subsequent chirps. During the estimation period the specified bins (defined as [negativeBinIdx, positiveBinIdx]) around DC are accumulated and averaged. It is assumed that no objects are present in the vicinity of the radar at that time. This procedure is initiated by the following CLI command, and it can be initiated any time while radar is running. Note that the maximum number of compensated bins is 32. The values in this command can be changed between sensorStop and sensorStart and even when the sensor is running. This is a mandatory command.		
		<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 whereas for advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
		<enabled> Enable DC removal using first few chirps 0 - disabled 1 - enabled	supported
		<negativeBinIdx> negative Bin Index (to remove DC from farthest range bins) Maximum negative range FFT index to be included for compensation. Negative indices are indices wrapped around from far end of 1D FFT. Ex: Value of -5 means last 5 bins starting from the farthest bin	supported
		<positiveBinIdx> positive Bin Index (to remove DC from closest range bins) Maximum positive range FFT index to be included for compensation Value of 8 means first 9 bins (including bin#0)	supported

		<numAvg> number of chirps to average to collect DC signature (which will then be applied to all chirps beyond this). Value of 256 means first 256 chirps (after command is issued and feature is enabled) will be used for collecting (averaging) DC signature in the bins specified above. From 257th chirp, the collected DC signature will be removed from every chirp.	The value must be power of 2, and must b greater than the numbe of Doppler bins.
clutterRemoval	Static clutter removal config message to datapath. Static clutter removal algorithm implemented by subtracting from the samples the mean value of the input samples to the 2D-FFT The values in this command can be changed between sensorStop and sensorStart and even when the sensor is running. This is a mandatory command.		
		<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 whereas for advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
		<enabled> Enable static clutter removal technique 0 - disabled 1 - enabled	supported
aoaFovCfg	Command for datapath to filter out detected points outside the specified range in azimuth or elevation plane The values in this command can be changed between sensorStop and sensorStart and even when the sensor is running. This is a mandatory command.		
		<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 whereas for advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
		<minAzimuthDeg>	minimum azimuth angl (in degrees) that specifies the start of field of view
		<maxAzimuthDeg>	maximum azimuth angle (in degrees) that specifies the end of fiel of view
		<minElevationDeg>	minimum elevation angle (in degrees) that specifies the start of field of view
		<maxElevationDeg>	maximum elevation angle (in degrees) that specifies the end of fiel of view
cfarFovCfg	Command for datapath to filter out detected points outside the specified limits in the range direction or doppler direction The values in this command can be changed between sensorStop and sensorStart and even when the sensor is running. This is a mandatory command.		
		<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 whereas for advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
		<procDirection> Processing direction: 0 – point filtering in range direction 1 – point filtering in Doppler direction	both values supported but this command should be given twice - one for range direction and other for doppler direction

		<min (meters or m/s)> the units depends on the value for <procDirection> field above. meters for Range direction and meters/sec for Doppler direction	minimum limits for the range or doppler below which the detected points are filtered out
		<max (meters or m/s)> the units depends on the value for <procDirection> field above. meters for Range direction and meters/sec for Doppler direction	maximum limits for the range or doppler above which the detected points are filtered out
compRangeBiasAndRxChanPhase	Command for datapath to compensate for bias in the range estimation and receive channel gain and phase imperfections. Refer to the procedure mentioned here The values in this command can be changed between sensorStop and sensorStart and even when the sensor is running. This is a mandatory command.		
		<rangeBias> Compensation for range estimation bias in meters	supported
		<Re(0,0)> <Im(0,0)> <Re(0,1)> <Im(0,1)> ... <Re(0,R-1)> <Im(0,R-1)> <Re(1,0)> <Im(1,0)> ... <Re(T-1,R-1)> <Im(T-1,R-1)> Set of Complex value representing compensation for virtual Rx channel phase bias in Q15 format. Pairs of I and Q should be provided for all Tx and Rx antennas in the device	For xwr1843, xwr6843 and xwr6443 demos: 1 pairs of values should be provided here since the device has 4 Rx an 3 Tx (total of 12 virtual antennas). Note the sign reversal required for phase compensatio coefficients in xwr6443 demo running on IWR6843AOP device. For xwr1642 demo: 8 pairs of values should be provided here since the device has 4 Rx an 2 Tx (total of 8 virtual antennas)
measureRangeBiasAndRxChanPhase	Command for datapath to enable the measurement of the range bias and receive channel gain and phase imperfections. Refer to the procedure mentioned here The values in this command can be changed between sensorStop and sensorStart and even when the sensor is running. This is a mandatory command.		
		<enabled> 1 - enable measurement. This parameter should be enabled only using the profile_calibration.cfg profile in the mmW demo profiles directory 0 - disable measurement. This should be the value to use for all other profiles.	supported
		<targetDistance> distance in meters where strong reflector is located to be used as test object for measurement. This field is only used when measurement mode is enabled.	supported
		<searchWin> distance in meters of the search window around <targetDistance> where the peak will be searched	supported
extendedMaxVelocity	Velocity disambiguation config message to datapath. A simple technique for velocity disambiguation is implemented. It corrects target velocities up to (2*vmax). The output of this feature may not be reliable when two or more objects are present in the same range bin and are too close in azimuth plane. The values in this command can be changed between sensorStop and sensorStart and even when the sensor is running. This is a mandatory command.	<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 whereas for advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
		<enabled> Enable velocity disambiguation technique 0 - disabled 1 - enabled	supported. Only disabled is supported for xwr64xx demo running on IWR6843AOP device.

CQRxSatMonitor	Rx Saturation Monitoring config message for Chirp quality to RadarSS and datapath. See mmwavelink doxgen for details on rlRxSatMonConf_t. The enable/disable for this command is controlled via the "analogMonitor" CLI command. The values in this command can be changed between sensorStop and sensorStart. This is a mandatory command	<profile> Valid profile Id for this monitoring configuration. This profile ID should have a matching profileCfg.	any value as per mmwavelink doxygen but corresponding profileCfg should be defined
		<satMonSel> RX Saturation monitoring mode	any value as per mmwavelink doxygen
		<priSliceDuration> Duration of each slice, 1LSB=0.16us, range: 4 -number of ADC samples	any value as per mmwavelink doxygen
		<numSlices> primary + secondary slices ,range 1-127. Maximum primary slice is 64.	any value as per mmwavelink doxygen
		<rxChanMask> RX channel mask, 1 - Mask, 0 - unmask	any value as per mmwavelink doxygen
CQSigImgMonitor	Signal and image band energy Monitoring config message for Chirp quality to RadarSS and datapath. See mmwavelink doxgen for details on rlSigImgMonConf_t. The enable/disable for this command is controlled via the "analogMonitor" CLI command. The values in this command can be changed between sensorStop and sensorStart. This is a mandatory command	<profile> Valid profile Id for this monitoring configuraiton. This profile ID should have a matching profileCfg	any value as per mmwavelink doxygen but corresponding profileCfg should be defined
		<numSlices> primary + secondary slices, range 1-127. Maximum primary slice is 64.	any value as per mmwavelink doxygen
		<numSamplePerSlice> Possible range is 4 to "number of ADC samples" in the corresponding profileCfg. It must be an even number.	any value as per mmwavelink doxygen
analogMonitor	Controls the enable/disable of the various monitoring features supported in the demos. The values in this command can be changed between sensorStop and sensorStart. This is a mandatory command.	<rxSaturation> CQRxSatMonitor enable/disable 1:enable 0: disable	supported
		<sigImgBand> CQSigImgMonitor enable/disable 1:enable 0: disable	supported
lvdsStreamCfg	Enables the streaming of various data streams over LVDS lanes. When this feature is enabled, make sure chirpThreshold in adcbufCfg is set to 1. The values in this command can be changed between sensorStop and sensorStart. This is a mandatory command.	<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 whereas for advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
		<enableHeader> 0 - Disable HSI header for all active streams 1 - Enable HSI header for all active streams	supported

		<dataFmt> Controls HW streaming. Specifies the HW streaming data format. 0-HW STREAMING DISABLED 1-ADC 4-CP_ADC_CQ	supported When choosing CP_ADC_CQ, please ensure that CQRxSatMonitor and CQSigImgMonitor commands are provide with appropriate values and these monitors are enabled using analogMonitor command.
		<enableSW> 0 - Disable user data (SW session) 1 - Enable user data (SW session) <enableHeader> should be set to 1 when this field is enabled.	supported
bpmCfg	BPM MIMO configuration. Every frame should consist of alternating chirps with pattern TXA+TxB and TXA-TXB where TXA and TXB are two azimuth TX antennas. This is alternate configuration to TDM-MIMO scheme and provides SNR improvement by running 2Tx simultaneously. When using this scheme, user should enable both the azimuth TX in the chirpCfg. See profile_2d_bpm.cfg profile in the xwr16xx mmW demo profiles directory for example usage. This config is supported and mandatory only for demos that use Doppler DSP DPU (xwr16xx/xwr68xx). This config is not supported and is not needed for demos that use Doppler HWA DPU (xwr18xx/xwr64xx).	<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 whereas for advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
		<enabled> 0-Disabled 1-Enabled	supported
		<chirp0Idx> BPM enabled: If BPM is enabled in previous argument, this is the chirp index for the first BPM chirp. It will have phase 0 on both TX antennas (TXA+ , TXB+). Note that the chirpCfg command for this chirp index must have both TX antennas enabled. BPM disabled: If BPM is disabled, a BPM disable command (set phase to zero on both TX antennas) will be issued for the chirps in the range [chirp0Idx..chirp1Idx]	any value as per mmwavelink doxygen but corresponding chirpCfg should be defined
		<chirp1Idx> BPM enabled: If BPM is enabled, this is the chirp index for the second BPM chirp.It will have phase 0 on TXA and phase 180 on TXB (TXA+ , TXB-). Note that the chirpCfg command for this chirp index must have both TX antennas enabled. BPM disabled: If BPM is disabled, a BPM disable command (set phase to zero on both TX antennas) will be issued for the chirps in the range [chirp0Idx..chirp1Idx].	any value as per mmwavelink doxygen but corresponding chirpCfg should be defined

calibdata	Boot time RF calibration save/restore command. Provides user to either save the boot time RF calibration performed by the RadarSS onto the FLASH or to restore the previously saved RF calibration data from the FLASH and instruct RadarSS to not re-perform the boot-time calibration. User can either save or restore or perform neither operations. User is not allowed to simultaneous save and restore in a given boot sequence. xwr18xx/60 Ghz devices: Boot time phase shift calibration data is also saved along with all other calibration data. The values in this command should not change between sensorStop and sensorStart. Reboot the board to try config with different set of values in this command This is a mandatory command.	<save enable> 0 - Save enabled. Application will boot-up normally and configure the RadarSS to perform all applicable boot calibrations during mmWave_open. Once the calibrations are performed, application will retrieve the calibration data from RadarSS and save it to FLASH. User need to specify valid <flash offset> value. <restore enable> option should be set to 0. 1 - Save disabled.	supported
		<restore enable> 0 - Restore enabled. Application will check the FLASH for a valid calibration data section. If present, it will restore the data from FLASH and provide it to RadarSS while configuring it to skip any real-time boot calibrations and use provided calibration data. User need to specify valid <flash offset> value which was used during saving of calibration data. <save enable> option should be set to 0. 1 - Restore disabled.	supported
		<Flash offset> Address offset in the flash to be used while saving or restoring calibration data. Make sure the address doesn't overlap the location in FLASH where application images are stored and has enough space for saving rlCalibrationData_t and rlPhShiftCalibrationData_t (xwr18xx/60 Ghz devices only) This field is don't care if both save and restore are disabled	supported
compressCfg	Compression configuration. This command enables compression configuration (supported only in the xwr64xx_compression beta demo) of radar data cube along Rx channel and range bin dimensions (reduced L3 RAM consumption) in Range DPU, based on the configuration. It is subsequently decompressed and processed in Doppler DPU accordingly. This config is supported and mandatory only for demos that use Doppler HWA DPU (xwr18xx/xwr64xx). This config is not supported and is not needed for demos that use Doppler DSP DPU (xwr16xx/xwr68xx).	<subFrameIdx> subframe Index	For legacy mode, that field should be set to -1 whereas for advanced frame mode, it should be set to either the intended subframe number or -1 to apply same config to all subframes.
		<ratio> Compression ratio needed in radar data cube size. For eg., value of 0.25 will achieve 25% compression in actual radar cube size.	Supported values: 0.25 0.5 and 0.75. (i.e., 25%, 50% and 75%)
		<numRangeBins> Number of range bins needed per compressed block.	Supported values from 1 to 32 (maximum possible).
sensorStart	sensor Start command to RadarSS and datapath. Starts the sensor. This function triggers the transmission of the		

	frames as per the frame and chirp configuration. By default, this function also sends the configuration to the mmWave Front End and the processing chain. This is a mandatory command.	Optionally, user can provide an argument 'doReconfig' 0 - Skip reconfiguration and just start the sensor using already provided configuration. <any other value> - not supported	supported
sensorStop	sensor Stop command to RadarSS and datapath. Stops the sensor. If the sensor is running, it will stop the mmWave Front End and the processing chain. After the command is acknowledged, a new config can be provided and sensor can be restarted or sensor can be restarted without a new config (i.e. using old config). See 'sensorStart' command. This is mandatory before any reconfiguration is performed post sensorStart.		supported
flushCfg	This command should be issued after 'sensorStop' command to flush the old configuration and provide a new one. This is mandatory before any reconfiguration is performed post sensorStart.		
configDataPort	This is an optional command to change the baud rate of the DATA_port. By default, the baud rate is 921600. This command will be accepted only when sensor is in init state or stopped state i.e. between sensorStop and sensorStart. It is recommended to use this command outside of the CFG file so that PC tools can also be configured to accept data at the desired baud rate.	<baudrate> The new baud rate for the DATA_port. Any valid baud rate upto max of 3125000. Recommended values: 921600, 1843200, 3125000.	supported
		<ackPing> 0 - Do not send any bytes on data port 1- Send 16 bytes of value '0xFF' to ack/sync over the DATA_port (binary) after change to baud rate is applied.	supported
queryDemoStatus	This is an optional command that can be issued anytime to get the sensor state (0-init,1-opened,2-started,3-stopped) of the device and the current baud rate of the DATA_port. The response of this command is provided on the CLI port.		
idlePowerDown	This is an optional command and can be issued anytime to put the system into power down mode. This command runs each of the low power functions to power the device down into Idle Mode and leaves it there indefinitely. A hard reset to the device is required in order to provide power and resume functional mode. This command is useful when needing to confirm power figures. This command is supported only for xWR6843 devices. This command will be available only when the macro SYS_COMMON_XWR68XX_LOW_POWER_MODE_EN is defined and the libsleep utils library is used.	<subframeidx> always set to -1	supported
		<enDSPpowerdown> 1 enables DSP Power Domain Off, 0 disables	supported
		<enDSSclkgate> 1 enables DSS Clock Gating, 0 disables	supported
		<enMSSvclkgate> 1 enables MSS Clock Gating, 0 disables	supported
		<enBSSclkgate> 1 enables BSS Clock Gating, 0 disables (Note: Performed last at code level as discussed above)	supported

		<enRFpowerdown> 1 enables RF Power Down, 0 disables	supported
		<enAPLLpowerdown> 1 enables APLL Power Down, 0 disables	supported
		<enAPLLGPADCpowerdown> 1 enables APLL/GPADC Power Down, 0 disables	supported
		<componentMicroDelay> specifies a delay duration, in microseconds, between each successive power function	supported
		<idleModeMicroDelay> specifies a delay duration, in microseconds, after Idle Mode has been acheived but before device is powered up (if using idlePowerCycle)	not supported
idlePowerCycle	This is an optional command and can be issued anytime to put the system into idle mode. This command runs each of the low power functions to power the device down into Idle Mode then, after a user-specified delay, powers the device back up into functional mode and ready to accept CLI Commands. This command is useful when needing to confirm or determine device functionality. This command is supported only for xWR6843 devices. This command will be available only when the macro SYS_COMMON_XWR68XX_LOW_POWER_MODE_EN is defined and the libsleep utils library is used.	<subframeidx> always set to -1	supported
		<enDSPpowerdown> 1 enables DSP Power Domain Off, 0 disables	supported
		<enDSSclkgate> 1 enables DSS Clock Gating, 0 disables	supported
		<enMSSvclkgate> 1 enables MSS Clock Gating, 0 disables	supported
		<enBSSclkgate> 1 enables BSS Clock Gating, 0 disables (Note: Performed last at code level as discussed above)	supported
		<enRFpowerdown> 1 enables RF Power Down, 0 disables	supported
		<enAPLLpowerdown> 1 enables APLL Power Down, 0 disables	supported
		<enAPLLGPADCpowerdown> 1 enables APLL/GPADC Power Down, 0 disables	supported
		<componentMicroDelay> specifies a delay duration, in microseconds, between each successive power function	supported
		<idleModeMicroDelay> specifies a delay duration, in microseconds, after Idle Mode has been acheived but before device is powered up (if using idlePowerCycle)	supported
%		Any line starting with '%' character is considered as comment line and is skipped by the CLI parsing utility.	

Table 1: mmWave SDK Demos - CLI commands and parameters

Running the prebuilt unit test binaries (.xer4f and .xe674)

Unit tests for the drivers and components can be found in the respective test directory for that component. See section "mmWave SDK - TI components" for location of each component's test code. For example, UART test code that runs on TI RTOS is in mmwave_sdk_<ver>/packages/ti/drivers/uart/test/<platform>. In this test directory, you will find .xer4f and .xe674 files (either prebuilt or build as a part of instructions mentioned in "Building drivers/control components"). Follow the instructions in section "CCS development mode" to download and execute these unit tests via CCS.